

Java Servlets are programs that run on a Web or Application server and act as a middle layer between a requests coming from a Web browser or other HTTP client and databases or applications on the HTTP server.

The servlet is initialized by calling the **init()** method.

- The servlet calls **service()** method to process a client's request.
- The servlet is terminated by calling the **destroy()** method.
- Finally, servlet is garbage collected by the garbage collector of the JVM.

The init() Method

The init method is called only once. It is called only when the servlet is created, and not called for any user requests afterwards. So, it is used for one-time initializations, just as with the init method of applets.

The servlet is normally created when a user first invokes a URL corresponding to the servlet, but you can also specify that the servlet be loaded when the server is first started.

When a user invokes a servlet, a single instance of each servlet gets created, with each user request resulting in a new thread that is handed off to doGet or doPost as appropriate. The init() method simply creates or loads some data that will be used throughout the life of the servlet.

```
public void init() throws ServletException { // Initialization code... }
```

The service() Method

The service() method is the main method to perform the actual task. The servlet container (i.e. web server) calls the service() method to handle requests coming from the client(browsers) and to write the formatted response back to the client.

Each time the server receives a request for a servlet, the server spawns a new thread and calls service. The service() method checks the HTTP request type (GET, POST, PUT, DELETE, etc.) and calls doGet, doPost, doPut, doDelete, etc. methods as appropriate.

Here is the signature of this method –

```
public void service(ServletRequest request, ServletResponse response) throws ServletException, IOException { }
```

The destroy() Method

The destroy() method is called only once at the end of the life cycle of a servlet. This method gives your servlet a chance to close database connections, halt background threads, write cookie lists or hit counts to disk, and perform other such cleanup activities.

After the `destroy()` method is called, the servlet object is marked for garbage collection. The `destroy` method definition looks like this –

```
public void destroy() { // Finalization code... }
```

Mail API

The **JavaMail API** provides a platform-independent and protocol-independent framework to build mail and messaging applications. The **JavaMail API** provides a set of abstract classes defining objects that comprise a mail system. It is an optional package (standard extension) for reading, composing, and sending electronic messages.

The **JavaMail API** is a Java API used to send and receive email via SMTP, POP3, and IMAP. It provides a platform-independent and protocol-independent framework to build mail and messaging applications.

Sending the mail

```
import java.util.Properties;

import javax.mail.*;
import javax.mail.internet.*;

public class EmailSender {

    public static void sendEmail(String to, String subject, String body) {

        final String from = "your_email@example.com";

        final String password = "your_password";

        // SMTP server configuration

        Properties props = new Properties();

        props.put("mail.smtp.host", "smtp.example.com");

        props.put("mail.smtp.port", "587");

        props.put("mail.smtp.auth", "true");

        props.put("mail.smtp.starttls.enable", "true");

        // Create session

        Session session = Session.getInstance(props, new Authenticator() {

            protected PasswordAuthentication getPasswordAuthentication() {
```

```

        return new PasswordAuthentication(from, password);
    }
});
try {
    // Compose message
    Message message = new MimeMessage(session);
    message.setFrom(new InternetAddress(from));
    message.RecipientType.TO, InternetAddress.parse(to));
    message.setSubject(subject);
    message.setText(body);
    // Send message
    Transport.send(message);
    System.out.println("Email sent successfully!");
} catch (MessagingException e) {
    e.printStackTrace();
}
}
}

```

Receiving the mails

```

import java.util.Properties;
import javax.mail.*;

public class EmailReceiver {

    public static void receiveEmail() {

        final String user = "your_email@example.com";
        final String password = "your_password";
        Properties props = new Properties();
    }
}

```

```

props.put("mail.store.protocol", "imaps");

try {
    Session session = Session.getDefaultInstance(props, null);
    Store store = session.getStore("imaps");
    store.connect("imap.example.com", user, password);
    Folder inbox = store.getFolder("INBOX");
    inbox.open(Folder.READ_ONLY);
    Message[] messages = inbox.getMessages();
    for (Message message : messages) {
        System.out.println("Subject: " + message.getSubject());
        System.out.println("From: " + message.getFrom()[0]);
        System.out.println("Text: " + message.getContent().toString());
    }
    inbox.close(false);
    store.close();
} catch (Exception e) {
    e.printStackTrace();
}

```

deleting the mail

```
package com.tutorialspoint;
```

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Properties;

```

```

import javax.mail.Flags;
import javax.mail.Folder;

```

```
import javax.mail.Message;
import javax.mail.MessagingException;
import javax.mail.NoSuchProviderException;
import javax.mail.Session;
import javax.mail.Store;

public class DeleteEmail {

    public static void delete(String pop3Host, String storeType, String user,
        String password)
    {
        try
        {
            // get the session object
            Properties properties = new Properties();
            properties.put("mail.store.protocol", "pop3");
            properties.put("mail.pop3s.host", pop3Host);
            properties.put("mail.pop3s.port", "995");
            properties.put("mail.pop3.starttls.enable", "true");
            Session emailSession = Session.getDefaultInstance(properties);
            // emailSession.setDebug(true);

            // create the POP3 store object and connect with the pop server
            Store store = emailSession.getStore("pop3s");

            store.connect(pop3Host, user, password);
```

```

// create the folder object and open it
Folder emailFolder = store.getFolder("INBOX");
emailFolder.open(Folder.READ_WRITE);

BufferedReader reader = new BufferedReader(new InputStreamReader(
    System.in));

// retrieve the messages from the folder in an array and print it
Message[] messages = emailFolder.getMessages();
System.out.println("messages.length---" + messages.length);
for (int i = 0; i < messages.length; i++) {
    Message message = messages[i];
    System.out.println("-----");
    System.out.println("Email Number " + (i + 1));
    System.out.println("Subject: " + message.getSubject());
    System.out.println("From: " + message.getFrom()[0]);

    String subject = message.getSubject();
    System.out.print("Do you want to delete this message [y/n] ? ");
    String ans = reader.readLine();
    if ("Y".equals(ans) || "y".equals(ans)) {
        // set the DELETE flag to true
        message.setFlag(Flags.Flag.DELETED, true);
        System.out.println("Marked DELETE for message: " + subject);
    } else if ("n".equals(ans)) {
        break;
    }
}
}

```

```
// expunges the folder to remove messages which are marked deleted
emailFolder.close(true);
store.close();

} catch (NoSuchProviderException e) {
    e.printStackTrace();
} catch (MessagingException e) {
    e.printStackTrace();
} catch (IOException io) {
    io.printStackTrace();
}
}

} public static void main(String[] args) {
String host = "pop.gmail.com";// change accordingly
String mailStoreType = "pop3";
String username = "abc@gmail.com";// change accordingly
String password = "*****";// change accordingly

delete(host, mailStoreType, username, password);

}

}
```